DECK 10 OF 10 · FRC JAVA CRASH COURSE

Autonomous path planning libraries

What each one does in 2026, and how to choose

WPILib 2026 · Java · Command-based

OVERVIEW

What this deck covers

A path library makes your robot drive a planned curve by itself. This deck covers what it does, what it needs, and how to pick one.

- A path follower, in the terms from Deck 6.
- What a robot needs before any library can help.
- How PathPlanner, Choreo, BLine and WPILib’s tools differ.
- How to choose the next step for a team.

EVIDENCE

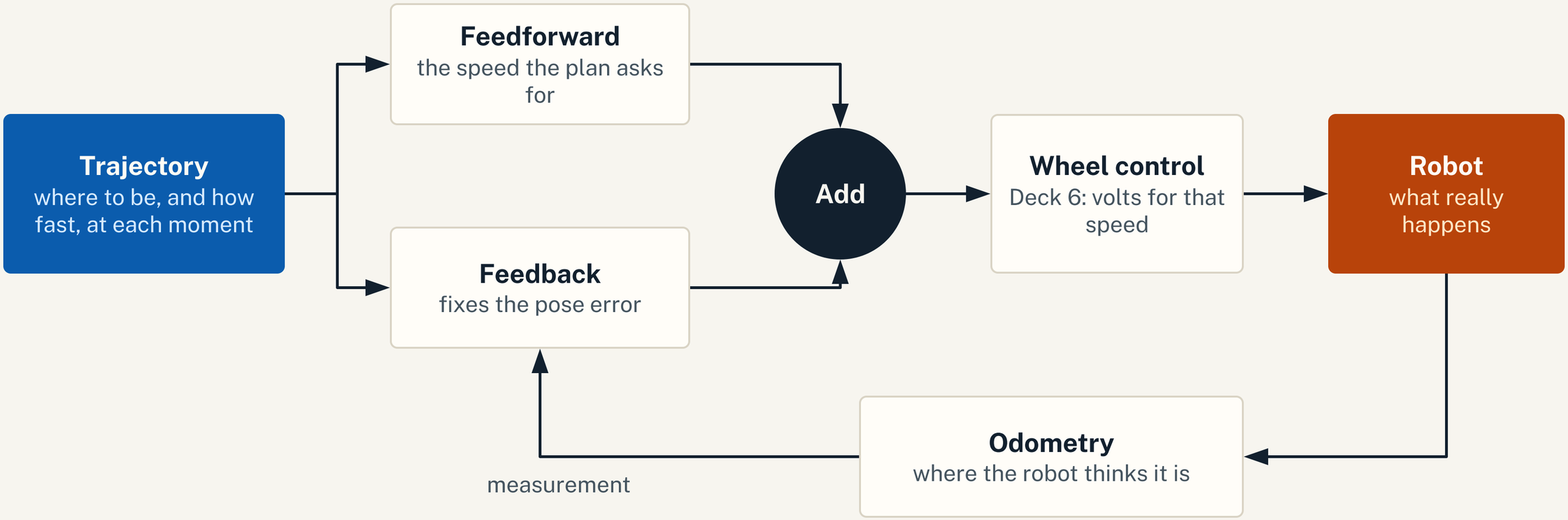
What 1,257 team repos from 2026 actually use

Library	Has the vendordep	Has its own path files
PathPlanner	887 repos (71%)	704 repos (56%)
Choreo	178 repos (14%)	127 repos (10%)
BLine	15 repos (1%)	14 repos (1%)
None of these three	309 repos (25%)	Not counted
WPILib’s own tools	No vendordep to count	Not counted

Our count, Nov 2026, and rough: public Java repos with “rebuilt” or “frc” in the name, made Nov 2025 to Apr 2026. It shows what is in a repo, not what ran at events.

CONTROL THEORY

What a path follower does 50 times a second



Error = plan - pose. A pose is a position on the field plus the way the robot faces.

WPILIB: BUILT IN, NO APP

The whole follower fits on one slide

deck10/WpilibPaths.java

```
  public static Command follow(
      CANDriveSubsystem drive, Trajectory path) {
1   var follower = new LTVUnicycleController(0.02);
    follower.setEnabled(USE_FEEDBACK);
    Timer clock = new Timer();
    return drive.startRun(() -> {
2       drive.resetPose(path.getInitialPose());
        clock.restart();
      }, () -> {
3       Trajectory.State plan = path.sample(clock.get());
        drive.driveSpeeds(
4           follower.calculate(drive.getPose(), plan));
      })
      .withTimeout(path.getTotalTimeSeconds())
      .finallyDo(drive::stop);
  }
```

1. **The follower**
   LTVUnicycleController is WPILib's follower for tank drives.

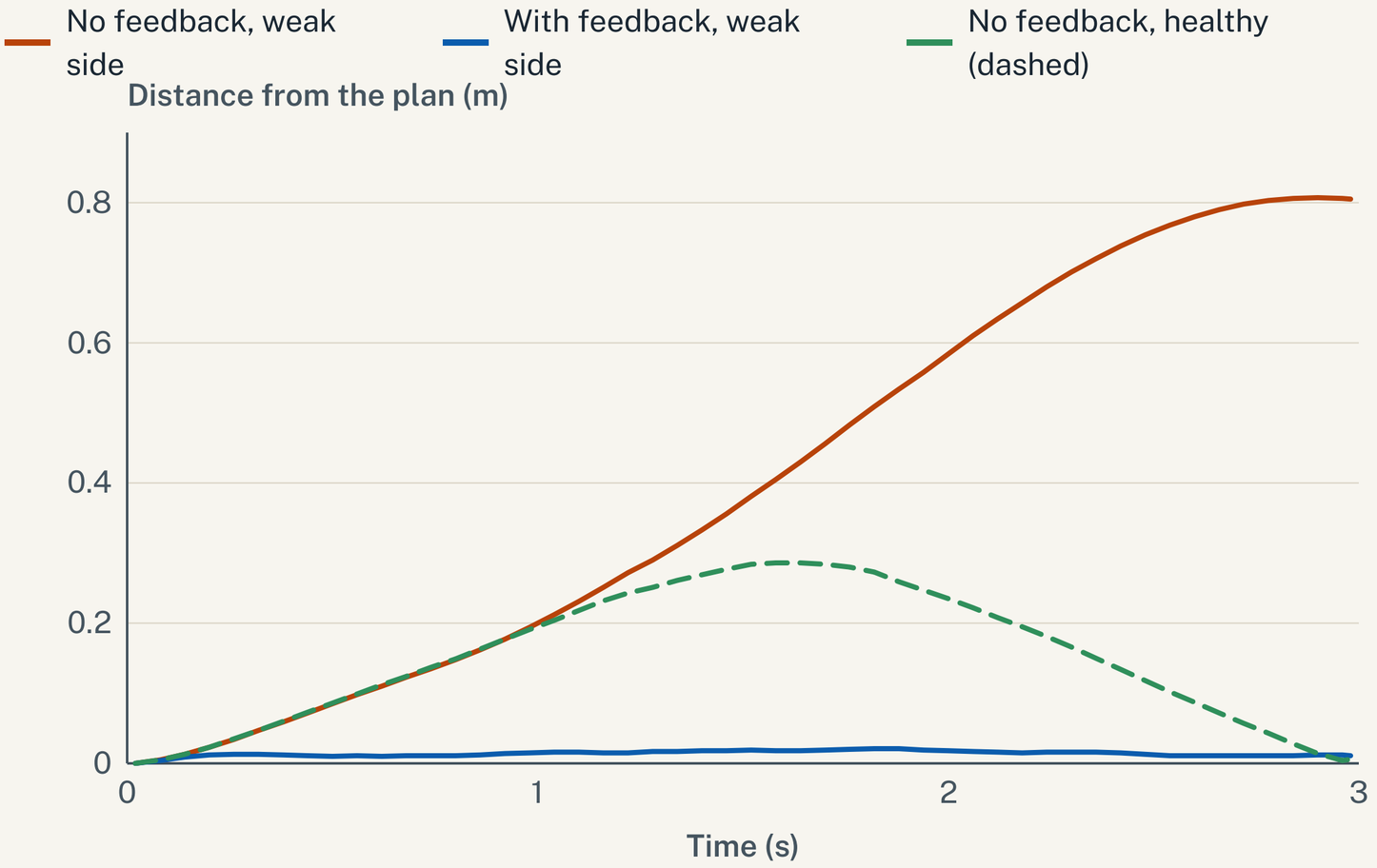
2. **Start at the start**
   Tell odometry the robot is at the first point.

3. **Sample the plan**
   The target pose and speed for this moment.

4. **Feedforward plus feedback**
   The planned speed, plus a fix for the pose error.

CONTROL THEORY

What feedback adds when the robot is not perfect



1. **Healthy, no feedback**
   Lags up to 29 cm, then catches up.

2. **Weak side, no feedback**
   Ends 81 cm off, turned 29 degrees.

3. **Weak side, with feedback**
   Never more than 2 cm off.

WHAT YOU NEED FIRST

Every library needs these, and a stock KitBot has none

You need	What it does	Stock 2026 KitBot
Wheel encoders	Measure how far each side rolled	None. CIMs have no built-in encoder
Gyro	Measures which way the robot points	None in the build or the code
Odometry	Turns those into a pose on the field	Nothing to feed it
Tuned drivetrain	Drives at the speed you ask for	Not yet: stick to volts, nothing measured

You can add them. The KitBot gearboxes have holes for optional encoders. Measure kV and kS with WPILib’s SysId tool.

BEST PRACTICE

Start with a timed auto

> **Get a timed auto working first. It needs no sensors and no library.**

FIRST’s KitBot code works this way. This one moved 2.5 m in our simulator, but carpet and battery change that, so measure on a real floor.

```
public static Command driveForward(CANDriveSubsystem drive) {
  return drive.driveArcade(() -> 0.6, () -> 0.0).withTimeout(1.5)
      .andThen(drive.runOnce(drive::stop))
      .andThen(Commands.print("launch fuel here"));
}
```

PATHPLANNER AND CHOREO

One lets you draw the path, one solves it

PathPlanner: you shape the curve

Drag **Bézier curves** in an app. That is a smooth line you bend with handles.

The timing is worked out on the robot, from your speed limits.

It can plan around obstacles during a match.

The follower comes with the library.

Choreo: a solver finds the fastest path

Place waypoints and limits in an app, and a solver does the rest.

The fastest path your drivetrain can do, saved to a fixed file.

Better robot numbers give better paths: mass, motors, gearing.

You write the follower yourself.

PATHPLANNER 2026.1.2

PathPlanner: set it up once, then run what you drew

deck10/PathPlannerPaths.java

```
  public static void configure(CANDriveSubsystem drive) {
    AutoBuilder.configure(
1       drive::getPose,
1       drive::resetPose,
1       drive::getSpeeds,
1       (speeds, feedforwards) -> drive.driveSpeeds(speeds),
2       pathFollower(),
        robotConfig(),
3       PathPlannerPaths::isRedAlliance,
        drive);
  }
  public static Command firstAuto() {
4   NamedCommands.registerCommand("launch", launch());
4   return new PathPlannerAuto("First Auto");
  }
```

1. **Four things you provide**
   Get pose, reset pose, read speeds, drive at speeds.

2. **Follower included**
   PPLTVController for tank. Swerve uses PPHolonomicDriveController.

3. **Draw on blue**
   It flips your paths to the red side when it should.

4. **Run what you drew**
   Register command names first, then load the auto.

CHOREO 2026.0.3

Choreo: you write the follower, then wire the routine

deck10/ChoreoPaths.java

```java
// You write the follower. Choreo hands you one sample per loop.
private void follow(DifferentialSample sample) {
  ChassisSpeeds plan = sample.getChassisSpeeds();
  drive.driveSpeeds(follower.calculate(drive.getPose(), sample.getPose(),
      plan.vxMetersPerSecond, plan.omegaRadiansPerSecond));
}

public AutoRoutine toHubAndLaunch() {
  AutoRoutine routine = factory.newRoutine("toHubAndLaunch");
  AutoTrajectory toHub = routine.trajectory("toHub"); // deploy/choreo/toHub.traj
  routine.active().onTrue(Commands.sequence(toHub.resetOdometry(), toHub.cmd()));
  toHub.atTime("spinUp").onTrue(Commands.print("spin up")); // a marker from the app
  toHub.done().onTrue(Commands.print("launch fuel here"));
  return routine;
}
```

BLINE 0.9.3: NEWEST OF THE FOUR

BLine: PID loops chase targets, with no timed plan

deck10/BLinePaths.java

```
  public static Command toHub(SwerveStandIn drive) {
    var follow = new FollowPath.Builder(
4       drive, drive::getPose,
        drive::getRobotRelativeSpeeds,
        drive::driveRobotRelative,
1       new PIDController(5.0, 0.0, 0.0), // translation
1       new PIDController(3.0, 0.0, 0.0), // rotation
1       new PIDController(2.0, 0.0, 0.0)) // cross-track
        .withPoseReset(drive::resetPose);
    FollowPath.registerEventTrigger("spinUp", spinUp());
    return follow.build(new Path(
2       new Path.Waypoint(2.0, 4.0, Rotation2d.kZero),
2       new Path.TranslationTarget(3.5, 4.5),
3       new Path.EventTrigger(0.5, "spinUp"),
2       new Path.Waypoint(5.0, 5.0, Rotation2d.kZero)));
  }
```

1. **Three PID loops**
   Speed along the path, heading, and staying on the line.

2. **Straight legs**
   The robot heads for each target, then hands off to the next.

3. **Events**
   They fire part way along a leg.

4. **Holonomic only**
   That means no tank. This code uses a swerve stand-in.

SIDE BY SIDE

The four options in 2026

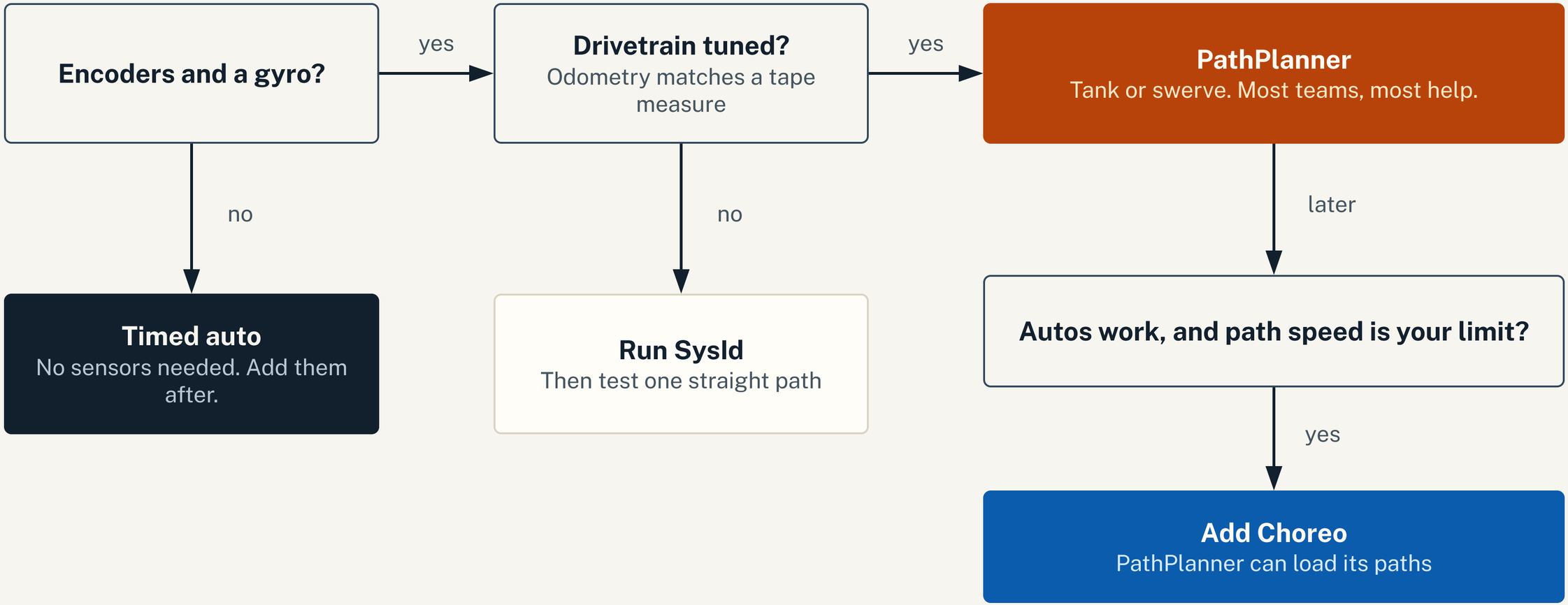
	PathPlanner	Choreo	BLine	WPILib tools
Paths are	Curves you draw	Solved for speed	Straight legs	Splines from code
Made in	App or code	App, before the match	App or code	Code
Drivetrains	Tank, swerve	Tank, swerve	Swerve, no tank	Tank; for swerve, docs point to others
During a match	Avoids obstacles, or paths from code	No new paths	Paths from code	Paths from code
Events	Markers, zones, triggers	Markers, triggers	Event triggers	None: use command groups
You write	Four methods	Pose, reset and the follower	Four methods, 3 PID loops	All of it
Made by	Team 3015; one main developer	SleipnirGroup; two main developers	Team 2638 student; one main developer	WPILib project

WPILib's older app, PathWeaver, is deprecated, so do not start with it.

2027: PathPlanner, Choreo and BLine all have 2027 work under way.

HOW TO CHOOSE

Start here: our recommendation for a young team



COMMON MISTAKE

The path is correct, but the start pose is wrong

What you see	The curve looks right and the logs say the robot hit its target, but the real robot misses.
Why it happens	Resetting odometry sets the pose to the value you give. The robot can’t detect a wrong placement.
The fix	Measure your starting pose, and line up the same way every match. In sim, 10° crooked ended 55 cm off.

EXERCISE

Follow a path, then add a fault

1. Run WPILib: Simulate Robot Code. In AdvantageScope, drag SimPose and TargetPose onto **Poses**.

2. Click **Autonomous** in Robot State. When the robot stops, read Path ended in the VS Code terminal.

3. In Constants, set LEFT_SIDE_STRENGTH to 0.9. Run it again and read the number.

4. Now also set USE_FEEDBACK to false. Run it again and read the number.

Done when: you have three end errors, one from each run.

RECAP

What this deck covered

- A follower is Deck 6 with a moving setpoint: feedforward from a plan, feedback on pose error.
- No encoders and gyro means no path following, so start with a timed auto.
- PathPlanner draws paths, Choreo solves them, BLine chases targets, and WPILib gives you the parts.
- For most young teams: timed auto, then PathPlanner, then Choreo when speed is your limit.

REFERENCE

Official docs

PathPlanner

pathplanner.dev

Choreo

choreo.autos

BLine

docs.bline.run

WPILib trajectories

docs.wpilib.org: Trajectory Tutorial